

MAKERERE



UNIVERSITY

P.O. Box 7062 Kampala Uganda
<https://www.cs.mak.ac.ug>

Telephone: 256-414-534560/1-9
E-mail: cs@cis.mak.ac.ug

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

Phishing Website Detection Using URL and Web Page Features: A Machine Learning Approach

by

MUJUNI INNOCENT

2400707155

24/U/07155/PS

innocent@ocunex.com

Department of Computer Science
School of Computing & Informatics Technology
Makerere University

A Research Proposal Submitted to the School of Computing and Information Technology in partial fulfillment of the requirements for the Award of the Degree of Bachelor of Science in Computer Science of Makerere University.

Supervisor

Dr. Arinaitwe Irene
irededats@gmail.com

April 2026

1.0 Background to the Study

The proliferation of internet-connected services over the past two decades has fundamentally changed how individuals, businesses, and governments operate. Online banking, mobile money, e-commerce, and cloud-based platforms have become central to everyday life, particularly across East Africa where internet penetration and digital financial inclusion are growing at a rapid pace. This expansion, however, has been accompanied by a parallel rise in cyber threats, of which phishing remains one of the most persistent and damaging.

Phishing is a form of social engineering attack in which fraudulent websites are designed to closely mimic legitimate platforms with the goal of deceiving users into surrendering sensitive information such as login credentials, credit card numbers, or personally identifiable data. The Anti-Phishing Working Group (APWG) detected over 1.2 million unique phishing sites in Q3 of 2023 alone, a 173% increase compared to the same period in 2022 [1]. These numbers reflect not just the scale of the problem but the increasing speed and sophistication with which attackers deploy and retire fraudulent infrastructure.

Early phishing attempts were relatively easy to identify: misspelled domains, absent HTTPS, and poorly reproduced page layouts were common indicators. Contemporary phishing sites are far more convincing. Attackers now routinely obtain valid SSL certificates to display the padlock icon in browsers, host phishing pages on legitimate cloud platforms to evade IP-based blocking, and use typosquatting techniques such as `paypa1.com` to register domains that appear credible at a casual glance.

The primary defensive mechanism deployed by browsers and security vendors has been black-listing. Services like Google Safe Browsing and PhishTank maintain regularly updated databases of known phishing URLs, and browsers query these lists before loading a page [2]. While effective against repeat offenders, blacklists are inherently reactive: they cannot flag a site that has not previously been reported. Research by Oest et al. [3] found that a large proportion of phishing victims are compromised within the first hour of a phishing site going live, well before blacklist coverage begins. Attackers exploit this window deliberately, often taking their infrastructure down within 24 hours.

Heuristic and rule-based detection systems attempted to bridge the zero-day gap by encoding known phishing patterns as explicit rules — for instance, flagging any URL that contains a numeric IP address, or any page whose form element submits data to an external domain. These systems

improved upon pure blacklisting but proved brittle in practice: once the detection rules were known, adversaries could craft pages that deliberately satisfied them.

Machine learning offers a more robust approach. Rather than relying on manually authored rules, supervised classifiers learn discriminative patterns directly from labelled datasets of phishing and legitimate websites. Sahingoz et al. [4] demonstrated that a Random Forest classifier trained on URL-based features alone could achieve 97.98% accuracy on a dataset of 73,000 URLs. Abdelnabi et al. [5] further showed that combining URL-level signals with web page content features substantially reduces false negatives on high-fidelity clone attacks, where the URL itself may look relatively clean but the page structure gives it away. This dual-feature approach forms the conceptual foundation of the present study, which proposes to develop and evaluate machine learning classifiers using features drawn from both URL strings and web page content.

2.0 Problem Statement

Despite significant research investment in phishing detection and the wide deployment of browser-integrated anti-phishing tools, phishing attacks continue to succeed at scale. The FBI’s Internet Crime Complaint Center (IC3) recorded losses exceeding USD 52 million directly attributable to phishing in 2022, and this figure does not account for downstream fraud, reputational damage to affected organisations, or the disproportionate impact on users with limited digital literacy [6]. The persistence of the threat, in the face of active countermeasures, reflects genuine unresolved deficiencies in current detection approaches.

Zero-day detection gap. Blacklist-based systems, which remain the most widely deployed anti-phishing mechanism, are fundamentally reactive. A phishing site that has not been previously reported is invisible to any blacklist-dependent tool. Since the average operational lifespan of a phishing site can be measured in hours, and since attackers deliberately launch and retire infrastructure within this window, blacklists consistently lag behind the threat. Users navigating to a newly deployed phishing site receive no warning until the URL has been identified, reported, reviewed, and propagated to the relevant browser or security tool — a process that can take far longer than the attack itself.

Dataset staleness and limited generalisability. Many of the machine learning models that dominate the published phishing detection literature were trained on datasets compiled several years ago. The UCI Phishing Websites Dataset, which remains one of the most frequently used benchmarks, was assembled in 2015 [7]. In the intervening decade, the phishing ecosystem has changed substantially: attackers now routinely use HTTPS to project an appearance of legitimacy, exploit legitimate cloud hosting services to evade domain-based blocking, and leverage browser-in-browser attack techniques that are not represented in older datasets [8]. A model trained on 2015-era features and attack patterns may therefore underperform against current threats, even if it achieves high accuracy on its original benchmark.

Fragmented feature usage. A review of the existing literature reveals that the majority of published studies evaluate machine learning classifiers on either URL-based features or web page content features, but rarely on a principled combination of both. This is a significant limitation. URL features are computationally inexpensive and do not require downloading the target page, but they can be evaded by attackers who construct carefully structured URLs. Web page content features provide complementary signal that is harder to manipulate without compromising the

attack’s effectiveness, but they impose greater extraction overhead. Treating these two feature classes as a unified input vector, as this study proposes, reflects how detection would realistically operate and is likely to produce more robust classifiers than either approach in isolation.

Reproducibility and transparency. A further concern is the reproducibility of published results. Many comparative studies in this domain do not publish their preprocessing pipelines, feature extraction code, or exact hyperparameter configurations, making it difficult for independent researchers to verify reported accuracy figures or build on existing work [9]. This limits the cumulative scientific value of the literature.

Contextual gap for East Africa. There is very limited published research evaluating phishing detection models in the context of East African internet infrastructure. Domain registration patterns, preferred hosting providers, popular target platforms (such as mobile money services), and regional certificate authority usage differ from the Western-centric environments that most benchmark datasets reflect. A model validated exclusively on Western phishing infrastructure may exhibit higher false negative rates when deployed against phishing campaigns targeting Ugandan or Kenyan users. This study acknowledges this gap and recommends future work on regionally representative dataset collection, even though a full empirical treatment of the regional dimension is beyond the scope of the current proposal.

Taken together, these gaps establish a clear case for a reproducible, well-documented study that combines URL and web page features within a rigorous comparative machine learning framework, applies this to current and well-curated datasets, and reports results in a manner that supports both practical deployment and future research.

3.0 Objectives

3.1 Main Objective

To develop a machine learning-based model for detecting phishing websites using a combined set of URL-based and web-page-based features, and to evaluate the model's classification performance using standard metrics.

3.2 Specific Objectives

1. To identify and justify the selection of key URL-based and web-page-based features that distinguish phishing websites from legitimate ones, through a systematic review of existing literature.
2. To collect, integrate, and preprocess publicly available phishing and legitimate website datasets suitable for supervised binary classification.
3. To implement and train three machine learning classifiers — Decision Tree, Random Forest, and Logistic Regression — on the prepared feature dataset.
4. To evaluate and compare the classification performance of each model using accuracy, precision, recall, F1-score, and ROC-AUC.
5. To determine the most discriminative features through feature importance analysis and interpret the contribution of each feature class to model performance.

4.0 Scope

Thematic Scope. This study is confined to the binary classification of web pages as either phishing or legitimate. It does not address related threat categories such as smishing (SMS-based phishing), vishing (voice phishing), spear phishing email campaigns, or other forms of social engineering that do not produce a web page artefact.

Technical Scope. Three classical supervised machine learning algorithms will be implemented and compared: Decision Tree, Random Forest, and Logistic Regression. Deep learning approaches — such as convolutional neural networks (CNNs), long short-term memory (LSTM) networks, and transformer-based architectures — are outside the scope of this study. This exclusion is deliberate: deep learning models require substantially greater computational resources, are considerably harder to interpret, and do not provide native feature importance outputs. In a security context where analysts need to understand why a specific site was flagged, classical machine learning is better suited. Feature extraction will be performed on pre-labelled datasets using programmatic URL parsing and HTML analysis; the study does not implement a live web crawler, real-time URL scanner, or browser-integrated detection tool.

Data Scope. The study relies exclusively on publicly available, pre-labelled datasets: the UCI Machine Learning Repository Phishing Websites Dataset [7], the Kaggle Phishing URL Dataset, and supplementary samples from PhishTank [10] and the Tranco top-site list [11]. All datasets are treated as static snapshots for the purpose of model training and evaluation. Real-time data feed integration is outside scope.

Geographical Scope. No geographical filter is applied to the datasets used. The models are intended to generalise broadly across phishing infrastructure. However, their performance against phishing campaigns specifically targeting East African users will not be empirically validated within this study, as no suitable region-specific labelled dataset is publicly available at this time.

Ethical Scope. All data sources used in this study are publicly available and contain no personally identifiable user information. The study does not involve human participants, live network traffic interception, or active probing of any third-party systems.

5.0 Justification

Phishing attacks impose tangible and growing costs on individuals, institutions, and the broader digital economy. The FBI’s IC3 reported direct phishing losses exceeding USD 52 million in 2022 [6], and that figure understates the true impact by excluding reputational damage, downstream identity fraud, and the cost of incident response. For users in Uganda and the wider East African region — where mobile money platforms, digital banking, and e-government services are increasingly central to daily life — effective phishing detection carries direct public interest value. A user deceived into submitting mobile money credentials to a fraudulent site may lose savings with limited recourse.

From a technical standpoint, this study addresses three specific shortcomings identified in the literature. First, it combines URL-based and web-page-based features into a unified feature representation, rather than treating them separately as most existing studies do. This approach better reflects how a deployed detection system would actually operate and is expected to produce a more robust classifier. Second, it applies systematic feature selection using information gain and chi-squared scoring, which produces an interpretable, reduced feature set that is practical for deployment in resource-constrained environments. Third, all experimental steps — dataset preprocessing, feature extraction, model training, and evaluation — will be fully documented and version-controlled, addressing the reproducibility gap that limits the practical utility of much of the existing literature [9].

The comparative evaluation of three classifiers — Decision Tree, Random Forest, and Logistic Regression — is also purposeful. Each algorithm offers a different balance of accuracy, interpretability, and calibration. Reporting results across all three, alongside feature importance scores, gives practitioners the information they need to choose an appropriate model for their specific deployment context, whether that is a browser extension, an email gateway, or a server-side URL screening service.

Finally, the study is grounded in the curriculum of the Bachelor of Science in Computer Science at Makerere University, demonstrating the practical application of machine learning, data preprocessing, and experimental evaluation to a problem with measurable social impact. The skills exercised — dataset curation, feature engineering, model development, and rigorous evaluation — are directly transferable to industry roles in cybersecurity, data science, and software engineering across the East African technology sector.

6.0 Literature Review

6.1 Detection Approaches: From Blacklists to Machine Learning

Phishing detection research has progressed through three broad generations: blacklist-based systems, heuristic classifiers, and machine learning models. Each generation addressed limitations of its predecessor while introducing new constraints.

Blacklists remain the most widely deployed anti-phishing mechanism. Services such as Google Safe Browsing, PhishTank, and Microsoft SmartScreen maintain databases of verified phishing URLs that browsers consult before loading a page [2]. Their precision against known threats is high, but they offer no protection against sites that have not yet been reported. Oest et al. [3] showed that many phishing victims are compromised within the first hour of a site going live, well before any blacklist entry is made. URL shorteners and redirect chains further undermine blacklisting by obscuring the true destination.

Heuristic systems attempted to address the zero-day gap through rule-based classifiers. Systems like CANTINA combined TF-IDF analysis of page text with WHOIS domain age lookups to flag suspicious pages without relying on prior knowledge of the URL. These approaches showed promise but were inherently brittle: as soon as the rules were documented, adversaries could craft pages that satisfied them while remaining malicious.

Machine learning fundamentally changed the detection paradigm. Rather than encoding rules manually, supervised classifiers learn discriminative patterns from labelled data and generalise to previously unseen examples. Mohammad et al. [12] constructed one of the most widely cited datasets in this domain — 11,055 instances with 30 URL and page features — and demonstrated that several classifiers could exceed 95% accuracy. Sahingoz et al. [4] extended this work with 73,000 URLs evaluated across seven algorithms, finding that Random Forest achieved 97.98% accuracy on URL features and that structural features combined with lexical n-gram representations consistently outperformed either in isolation.

More recent deep learning approaches have pushed accuracy higher still. Le et al. [13] proposed URLNet, a CNN that operates directly on raw URL character sequences without manual feature extraction, achieving 98.2% accuracy on 1.4 million URLs. Han et al. [8] applied transformer-based models to phishing detection on social platforms, achieving strong results on short, obfuscated URLs that confound classical approaches. Despite their accuracy, deep learning methods are com-

putationally expensive, lack interpretability, and do not produce native feature importance scores — properties that limit their suitability in a security context where analyst explainability matters. Abdelnabi et al. [5] demonstrated that combining URL features with page content features substantially reduces false negatives on high-fidelity clone attacks, reinforcing the dual-feature design of this study.

6.2 Feature Engineering

Feature selection is among the most consequential design decisions in any phishing detection system. The literature consistently identifies two complementary categories: URL-based features and web page content features. Table 1 presents the most widely validated features drawn from multiple studies [4, 12, 14].

Table 1: Key Phishing Detection Features

No.	Feature	Description	Phishing Signal
<i>URL-Based Features</i>			
1	URL Length	Total character count of URL	>75 chars is suspicious
2	IP Address in URL	Numeric IP instead of domain name	Strong phishing indicator
3	@ Symbol	Redirects browser to post-@ address	Strong phishing indicator
4	URL Shortening	Uses bit.ly or similar services	True destination is hidden
5	Hyphen in Domain	- present in domain name	Common in fake brand URLs
6	Sub-domain Count	Number of dots in host-name	>3 is suspicious
7	HTTPS Protocol	SSL certificate present	Absence is suspicious
8	Domain Age	Months since domain was registered	<6 months is suspicious
9	Favicon Domain	Favicon loaded from external domain	Mismatch indicates phishing
10	Non-standard Port	Unusual port number in URL	Indicates phishing
<i>Web Page / Content Features</i>			
11	External Object Ratio	% of resources from other domains	>61% is suspicious
12	Anchor Tag URLs	% of links pointing off-domain	High ratio is suspicious
13	Form Action Domain	Form submits to an external domain	Strong phishing indicator
14	Hidden Iframes	Zero-size or hidden iframe tags	Common in phishing pages
15	DNS Record	Valid DNS record exists for domain	Absence indicates phishing
16	Web Traffic Rank	Tranco ranking of domain	Very low rank is suspicious
17	Google Index Status	Whether Google indexes the page	Not indexed is suspicious
18	Backlink Count	External links pointing to page	Very few is suspicious
19	Statistical Report	Flagged by PhishTank or similar	Direct phishing indicator

URL features are computationally inexpensive and require no page download, making them suitable for a first-pass filter. The IP-address-in-URL feature is particularly reliable: phishing

infrastructure is frequently deployed on bare IP addresses to permit rapid rotation and avoid DNS-based takedown. The @ symbol feature exploits a browser parsing convention where everything before @ is treated as authentication credentials, so `http://paypal.com@evil.com` silently navigates to `evil.com`. Page content features catch what URL analysis misses: a form submitting credentials to an external domain is almost always indicative of a credential harvesting attack, and a page assembled almost entirely from externally hosted resources is a common pattern in copy-paste phishing [12]. WHOIS-derived features such as domain age are strong signals but introduce API latency in live systems; this study uses pre-extracted versions from the UCI dataset.

6.3 Machine Learning Algorithms

Decision Tree classifiers recursively partition the feature space using binary thresholds chosen to maximise class separation, measured by Gini impurity or information gain. Their principal advantage in a security context is interpretability: the resulting tree structure is visualisable and every prediction path is traceable. The main limitation is a tendency to overfit on noisy training data when the tree depth is unconstrained.

Random Forest builds an ensemble of decision trees, each trained on a bootstrapped sample of data with a random subset of features considered at each split, with final predictions made by majority vote. This randomisation substantially reduces variance while maintaining low bias. Sahingoz et al. [4] and Rao and Pais [14] both found Random Forest to be the highest-performing classical classifier in their respective phishing experiments. It also natively produces feature importance scores via mean decrease in impurity, which this study uses for post-hoc analysis.

Logistic Regression models the log-odds of class membership as a linear combination of input features, producing calibrated class probabilities rather than hard labels. This gives practitioners control over the precision-recall trade-off by adjusting the decision threshold. The learned coefficients are directly interpretable, and the model is generally more robust to class imbalance than tree-based methods when paired with L2 regularisation [5].

6.4 Research Gaps

Several gaps in the existing literature directly motivate this study. First, the most widely cited phishing datasets, including the UCI dataset used here, were compiled around 2015 and do not reflect current attacker techniques such as HTTPS phishing or cloud-hosted infrastructure [8]. Second, reproducibility is an ongoing concern: most comparative studies do not publish their full

preprocessing pipelines, making results difficult to verify or build upon [9]. Third, the majority of published studies treat URL features and page content features as separate inputs rather than combining them into a unified feature vector, which limits the generalisability and practical applicability of their findings. This study addresses all three gaps.

7.0 Methodology

This study adopts a quantitative experimental research design. Three supervised machine learning classifiers are trained on labelled phishing and legitimate website data and evaluated using standard binary classification metrics. The design follows an established pipeline in applied machine learning research, summarised in Figure 1.

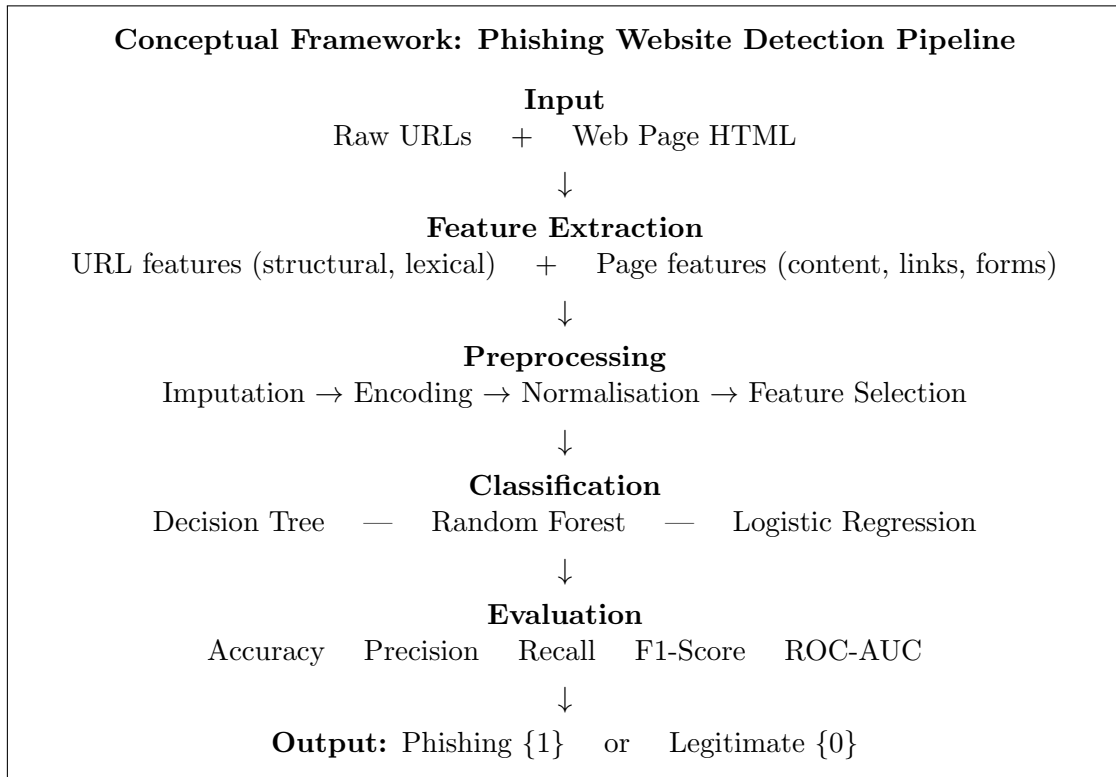


Figure 1: Conceptual framework for machine learning-based phishing website detection.

The framework treats phishing detection as a supervised binary classification problem. URL and page features are the independent variables; the binary class label (phishing or legitimate) is the dependent variable. The core assumption — that phishing and legitimate sites differ in statistically learnable ways across these features — is well-supported by the literature [4, 12].

7.1 Research Approach

The research follows a quantitative experimental approach, measuring and comparing classifier performance under controlled, reproducible conditions. The pipeline runs in six stages: (1) dataset acquisition and integration; (2) exploratory data analysis (EDA); (3) preprocessing and feature

engineering; (4) model training with cross-validated hyperparameter tuning; (5) evaluation on a held-out test set; and (6) feature importance analysis. All code will be implemented in Python using Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn, and version-controlled with Git.

7.2 Data Collection Procedure

Four publicly available data sources will be used.

UCI Phishing Websites Dataset [7]: 11,055 instances with 30 pre-extracted URL and page features labelled as phishing or legitimate. Features are encoded as categorical signals $(-1, 0, 1)$, making this the primary structured dataset and enabling direct comparison with prior work.

Kaggle Phishing URL Dataset: Over 88,000 raw labelled URLs. Unlike UCI, this dataset requires manual feature extraction using Python’s `urllib` and `re` libraries and provides a larger, more recent complement to the UCI data.

PhishTank [10]: A community-verified feed of verified phishing URLs. A snapshot will be collected to enrich the phishing class with recent examples not represented in the UCI dataset.

Tranco Top Site List [11]: A research-grade ranking of high-traffic legitimate websites. Top-ranked sites will be sampled for the legitimate class, as established high-traffic sites are almost never phishing targets.

The combined dataset will be balanced to an approximate 1:1 phishing-to-legitimate ratio via stratified sampling, yielding an estimated 15,000–20,000 instances for modelling.

7.3 Data Analysis Procedure

Exploratory Data Analysis: Prior to modelling, EDA will examine class distribution, per-feature value distributions, and a Spearman correlation matrix to identify redundant or near-constant features.

Preprocessing: UCI features will be treated as ordinal integers for tree-based models and one-hot encoded for logistic regression. Missing values (zero-encoded in the UCI dataset) will be imputed using per-feature training-fold modes. Page features from the Kaggle dataset will be extracted from cached HTML using BeautifulSoup.

Feature Selection: Information gain and chi-squared scoring will rank all features. The top 20 by combined score will be retained for modelling, following Wei et al. [15], who showed that a pruned 20-feature Random Forest matched a full 87-feature model with significantly faster inference.

Model Training: Three classifiers will be trained using Scikit-learn:

1. **Decision Tree:** Gini impurity as split criterion. Maximum depth and minimum samples per leaf tuned via 5-fold stratified grid search.
2. **Random Forest:** 100 estimators; number of features per split and maximum tree depth tuned by the same cross-validation procedure. Feature importances extracted post-training via mean decrease in impurity.
3. **Logistic Regression:** L2 regularisation. Regularisation strength C and solver (liblinear, lbfgs) selected by cross-validation. Coefficient magnitudes reported for interpretability.

7.4 Parameters Used to Judge Accuracy

Five metrics will be used to evaluate and compare each model.

Accuracy measures the proportion of correctly classified instances:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy is reported but not used alone, as it can be misleading when class distributions are imbalanced.

Precision and **Recall** capture the two error types of primary concern — false alarms and missed phishing sites:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

F1-Score is the harmonic mean of precision and recall, providing a single balanced measure:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC-AUC (Area Under the Receiver Operating Characteristic Curve) evaluates class separation across all possible decision thresholds, making it robust to class imbalance. A score of 1.0 indicates perfect separation; 0.5 is equivalent to random guessing [16]. A confusion matrix will also be reported for each model to make the distribution of error types explicit.

7.5 Data Preparation, Training, and Testing

The dataset will be partitioned into 80% training and 20% test sets using stratified random sampling to preserve the class ratio in both partitions. The test set is held out entirely and is not used at any point during model training or hyperparameter selection.

Hyperparameter tuning is performed exclusively within the training partition using 5-fold stratified cross-validation. Within each fold, preprocessing steps are applied in sequence — imputation using training-fold statistics, standardisation for logistic regression (tree-based models are scale-invariant), and feature selection scored on the training fold only — all encapsulated in Scikit-learn Pipeline objects to prevent data leakage. Final performance metrics are reported on the held-out test set only.

7.6 Ethical Considerations

This study raises no significant human subjects concerns. The data consists entirely of URLs and web page content from publicly available datasets, none of which contain personally identifiable information. No human participants are involved, no informed consent is required, and no institutional ethics committee submission is necessary. All four datasets — UCI, Kaggle, PhishTank, and Tranco — are publicly released for research use and will be used only as described in this proposal.

One issue that warrants attention is dual-use risk. Reporting which specific feature patterns are associated with misclassification could assist attackers in refining evasion techniques. To mitigate this, feature importance results will be reported at an aggregate level sufficient for defensive insight, without characterising individual misclassified instances in a way that constitutes an evasion guide. This is consistent with responsible disclosure norms in security research [17].

A fairness consideration also applies. Datasets compiled from Western web infrastructure may not accurately represent phishing targeting East African users, which could lead to elevated false negative rates in that regional context. This limitation will be clearly acknowledged in the study’s discussion, along with a recommendation for future work on geographically representative dataset collection. Finally, all code, random seeds, and hyperparameter configurations will be published in a public repository upon completion of the study, in line with reproducibility standards for machine learning research [9].

References

- [1] Anti-Phishing Working Group, “Phishing Activity Trends Report, 3rd Quarter 2023,” <https://apwg.org/trendsreports/>, 2023, accessed: Apr. 2026.
- [2] Google LLC, “Google Safe Browsing API,” <https://developers.google.com/safe-browsing>, 2023, accessed: Apr. 2026.
- [3] A. Oest, Y. Safaei, P. Zhang, B. Wardman, G. Warner, and G.-J. Ahn, “PhishTime: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists,” in *Proc. 29th USENIX Security Symposium*. USENIX, 2020, pp. 379–396.
- [4] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [5] S. Abdelnabi, K. Krombholz, and M. Fritz, “VisualPhishNet: Zero-day phishing website detection by visual similarity,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*. ACM, 2020, pp. 1681–1698.
- [6] Federal Bureau of Investigation, “Internet Crime Report 2022,” https://www.ic3.gov/Media/PDF/AnnualReport/2022_IC3Report.pdf, 2022, accessed: Apr. 2026.
- [7] R. M. Mohammad, F. Thabtah, and T. L. McCluskey, “UCI machine learning repository: Phishing websites data set,” <https://archive.ics.uci.edu/ml/datasets/Phishing+Websites>, 2015, accessed: Apr. 2026.
- [8] W. Han, Y. Cao, E. Bertino, and J. Yong, “Using automated individual white-hat social engineering attacks to prevent social engineering attacks,” *Computers & Security*, vol. 114, p. 102579, 2022.
- [9] O. E. Gundersen and S. Kjetsmo, “State of the art: Reproducibility in artificial intelligence,” *Proc. AAAI Conf. on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [10] PhishTank, “PhishTank Developer Information,” https://www.phishtank.com/developer_info.php, 2023, accessed: Apr. 2026.
- [11] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczynski, and W. Joosen, “Tranco: A research-oriented top sites ranking hardened against manipulation,” *arXiv preprint arXiv:1806.01156*, 2019.
- [12] R. M. Mohammad, F. Thabtah, and T. L. McCluskey, “Predicting phishing websites based on self-structuring neural network,” *Neural Computing and Applications*, vol. 25, no. 2, pp. 443–458, 2014.
- [13] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL representation with deep learning for malicious URL detection,” in *arXiv preprint arXiv:1802.03162*, 2018.
- [14] R. S. Rao and A. R. Pais, “Detection of phishing websites using an efficient feature-based machine learning framework,” *Neural Computing and Applications*, vol. 31, pp. 3851–3873, 2019.
- [15] B. Wei, R. A. Hamad, L. Yang, X. He, H. Wang, B. Gao, and W. L. Woo, “A deep-learning-driven light-weight phishing detection sensor,” *Sensors*, vol. 19, no. 19, p. 4258, 2020.
- [16] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [17] R. Anderson and T. Moore, “Information security economics and beyond,” *Communications of the ACM*, vol. 63, no. 5, pp. 33–35, 2020.

Appendix A: Budget

No.	Activity / Item	Qty.	Cost (UGX)
1	Internet data (dataset download, cloud storage, literature access)	1 month	120,000
2	Printing and binding (proposal and final report)	3 copies	45,000
3	Stationery (notebooks, pens, flash drives)	Lump sum	30,000
4	Transport (library visits and supervisor meetings)	10 trips	100,000
5	Google Colab Pro (compute for model training)	1 month	60,000
6	Report typesetting and formatting	Lump sum	50,000
7	Contingency (10%)	Lump sum	40,500
Total Estimated Budget			445,500

Appendix B: Work Plan / Timeline (1 Month)

No.	Activity	Duration
1	Literature review, conceptual framework, and proposal refinement	Days 1–5
2	Dataset acquisition, integration, and exploratory data analysis	Days 6–8
3	Preprocessing, feature extraction, and feature selection	Days 9–11
4	Model implementation, cross-validation, and hyperparameter tuning	Days 12–17
5	Model evaluation, comparison, and feature importance analysis	Days 18–20
6	Results interpretation and report writing	Days 21–26
7	Final review, submission, and presentation preparation	Days 27–30